

KARMA

Kubernetes Attack & Remediation Mapping Agent

Controlled Attack Validation, Detection and AI-Assisted Recovery

Technical Documentation

Ritvik Indupuri

Abstract

KARMA is a Kubernetes security lab that creates a local Kind cluster, executes ten controlled attack scenarios, watches the cluster for security-relevant resource changes and maps activity to MITRE ATT&CK; for Containers. Operators can run individual scenarios, launch the full catalog, inspect live events and generate assessment evidence from the command line or FastAPI service.

A Claude Sonnet 4 remediation agent can receive high- or critical-severity incident context, stream a remediation plan and execute parsed kubectl commands against the lab. The platform also includes a model-backed security chat, React monitoring interface, WebSocket event stream and ReportLab PDF generator.

Document scope

- Kind lab architecture, control plane and trust boundaries.
- Attack execution, detection and AI-remediation lifecycle.
- Ten attack modules and MITRE ATT&CK; mapping.
- Cluster management, event delivery, API, CLI and reporting.
- Operational safeguards, deployment and implementation cross-reference.

Contents

Abstract	2
1 System overview	3
2 System architecture	4
3 Attack and remediation lifecycle	6
4 Attack engine and catalog	7
5 Cluster management	8
6 Detection and event pipeline	9
7 Claude remediation agent	9
8 CLI, API and security chat	10
9 Reporting and MITRE coverage	10
10 Deployment and safe operation	11
11 Implementation cross-reference	11
12 Next steps	12

1 System overview

KARMA is designed for an isolated, intentionally vulnerable test environment. It uses the Kubernetes Python client and kubectl to create real resources and observe real cluster behavior; it is not a passive simulator. The supplied cluster name is k8s-attack-lab and the workflow can create or reuse the lab before an engagement.

Use KARMA only on infrastructure you own or are explicitly authorized to test. The attack modules and remediation agent can create, modify and delete Kubernetes resources.

1.1 Functional layers

Layer	Responsibility
Operator	Interactive terminal, REST API and React dashboard.
Control plane	Cluster lifecycle, attack orchestration, detection, remediation and event streaming.
Security lab	Three-node Kind cluster and intentionally vulnerable Kubernetes resources.
Intelligence	MITRE coverage, detection alerts, Claude remediation and security chat.
Evidence	Session JSON, live timelines, state summaries and PDF assessment reports.

1.2 Core technology

Area	Technology
Backend	Python 3.11+, FastAPI, Uvicorn, asyncio
Cluster	Docker, Kind, kubectl, Kubernetes Python client
Model services	Anthropic SDK, Claude Sonnet 4
Frontend	React, TypeScript, Vite, Tailwind CSS
Reporting	ReportLab; JSON result files

2 System architecture

Figure 1 separates operator surfaces, KARMA services, the Kind lab and evidence outputs. Only the two Claude-backed roles use agent icons; attack classes, routers and infrastructure remain ordinary components.

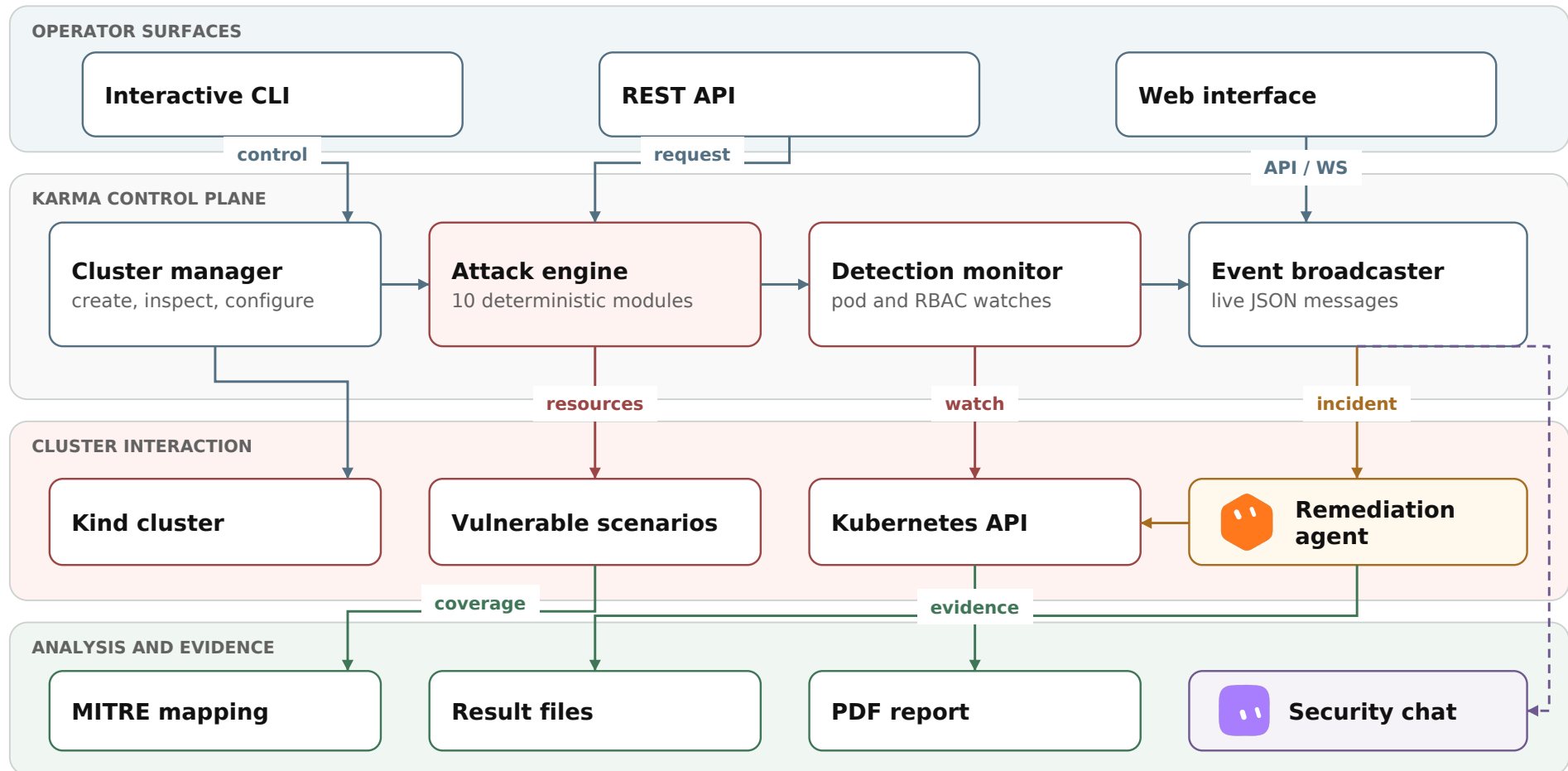


Figure 1: KARMA logical architecture and cluster interaction paths.

2.1 Operator and control surfaces

KARMA exposes the same lab through an interactive CLI, a FastAPI service and a React web interface. The web interface receives live JSON events over WebSocket, while the API and CLI invoke the cluster manager, attack engine and orchestration services directly. These entry points coordinate the platform but do not themselves implement attack behavior.

2.2 Lab execution boundary

The Kind cluster is an intentionally vulnerable local Kubernetes environment. Attack modules create pods, RBAC bindings, service accounts and workload conditions through the Kubernetes Python client or `kubect1`. The detection monitor watches pod and RBAC activity, and the event broadcaster publishes commands, outputs, affected infrastructure and status changes to connected clients.

2.3 Agent-backed capabilities

The remediation agent receives high- or critical-severity incident context when an Anthropic API key is configured. Claude Sonnet 4 generates remediation commands, the backend executes supported `kubect1` steps and command results are broadcast to the interface and stored in the remediation session. Security chat is the second Claude-backed role and answers questions using current platform state. Only these two roles use agent icons.

2.4 Evidence outputs

Attack metadata is mapped to MITRE ATT&CK tactics and techniques. Session JSON, command results, detection events and affected-resource records support the downloadable assessment report. The architecture therefore separates interactive control, destructive lab execution, model-assisted recovery and evidence generation into explicit boundaries.

3 Attack and remediation lifecycle

Figure 2 shows a full engagement from readiness checks through attack execution, watch-based detection, conditional remediation and evidence production. The remediation branch is used for high- and critical-severity incidents when an Anthropic API key is configured.

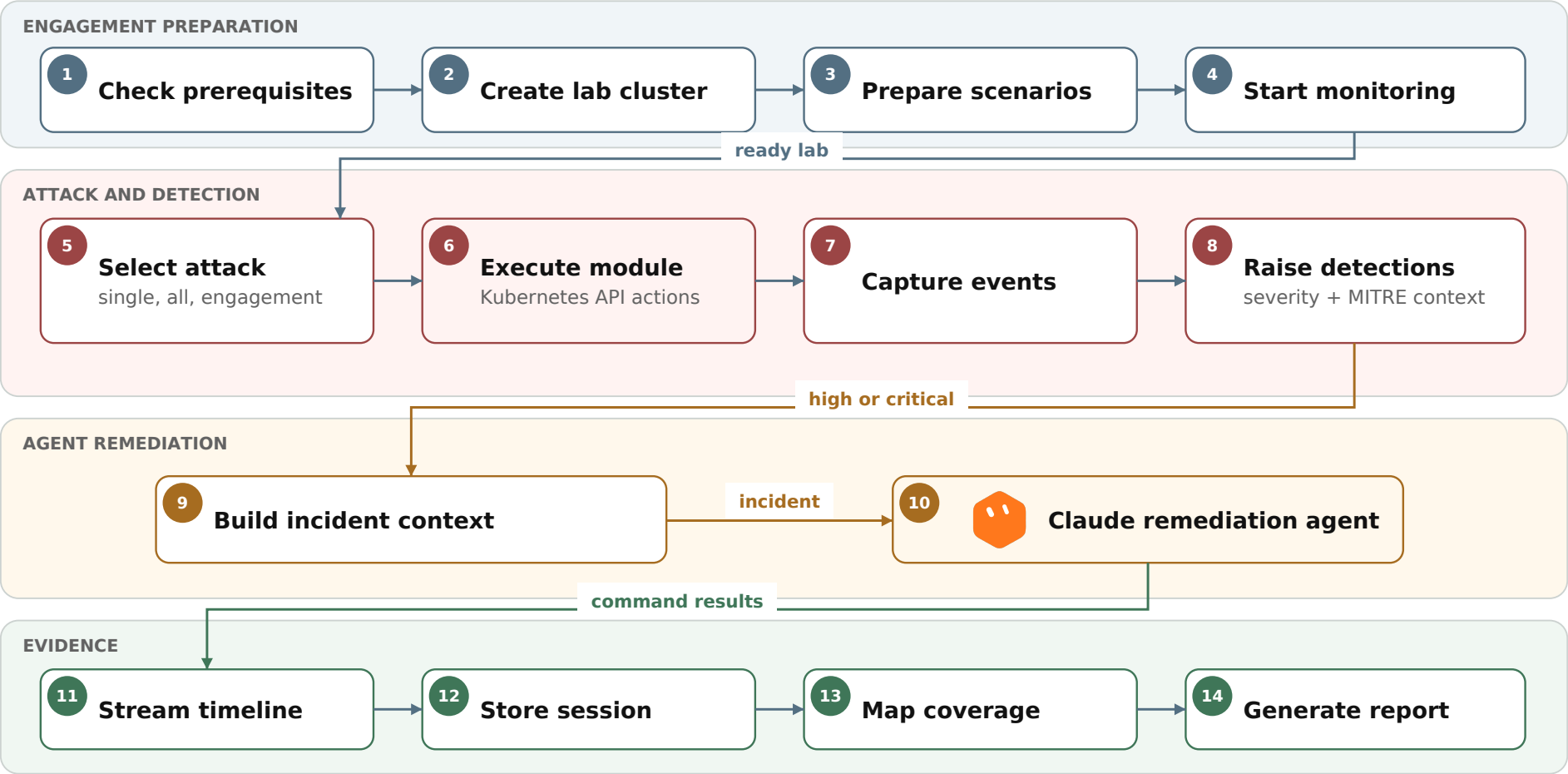


Figure 2: Controlled engagement lifecycle from lab preparation to report evidence.

3.1 Preparation and attack execution

KARMA checks Docker, Kind and `kubect1`, creates or reuses the three-node lab and prepares the deliberately weak scenario resources. The operator can launch one attack, the full catalog or a sequenced engagement. Each module records its execution identifier, command or API activity, timestamps, output and affected infrastructure while continuing asynchronously under the attack engine.

3.2 Detection and conditional remediation

The detection monitor observes pod and RBAC changes while the event pipeline sends progress to the terminal and web interface. When an attack completes, the backend assembles the attack record with recent detection events. High- and critical-severity incidents enter the remediation queue only when the Claude-backed remediation service is available; lower-severity results remain evidence without triggering that branch.

For a queued incident, the remediation agent analyzes the attack and infrastructure context, emits its reasoning stream, extracts complete `kubect1` command blocks and executes them as they arrive. This uses one model request; command results are not fed back into a subsequent Claude turn. Individual command results are returned to the active remediation session so the operator can see both the proposed recovery and its execution outcome.

3.3 Evidence completion

The platform streams the combined timeline, stores session results, maps the exercised behavior to MITRE ATT&CK and generates the PDF report. In the diagram, the single agent icon marks the model-backed remediation stage; readiness checks, attack modules, monitoring, evidence mapping and reporting are deterministic platform operations.

4 Attack engine and catalog

AttackEngine registers ten classes, launches one attack in a background task, tracks active executions and retains the latest 100 in-memory results. Every module inherits a common base that records status, timestamps, affected infrastructure and emitted events.

4.1 Implemented scenarios

#	Scenario	Severity	Repository technique IDs
1	HostPath privilege escalation	Critical	T1611
2	RBAC privilege escalation	Critical	T1548.003
3	Privileged container escape	Critical	T1611
4	Sidecar proxy injection	High	T1613; T1021.006
5	Kubernetes secrets exfiltration	Critical	T1552.007
6	ConfigMap data collection	Medium	T1613
7	Internal network scan	High	T1046
8	Kubelet API abuse	Critical	T1611; T1609
9	Cluster resource hijacking	High	T1496; T1499
10	DNS-based exfiltration	High	T1048; T1572

4.2 Execution characteristics

Modules create actual pods, bindings, service accounts or workload conditions in the lab. Their events include the command or API operation, output and infrastructure affected. The run-all orchestrator launches the catalog in sequence with a two-second interval; individual executions continue asynchronously under AttackEngine.

5 Cluster management

ClusterManager checks Docker, Kind and kubectl, creates a three-node Kind topology, loads Kubernetes client configuration and returns structured cluster information. Scenario setup labels the default namespace to allow the deliberately privileged exercises and provisions vulnerable resources used by the attacks.

5.1 Managed state

Object	Collected or controlled information
Nodes	Readiness, kubelet version, operating system, architecture, IP and capacity.
Pods	Namespace, node assignment, phase, IP and container count.
Namespaces	Available namespaces and scenario context.
Services	Cluster IP, type and port mappings.
Lab lifecycle	Prerequisite checks, create, reuse, scenario setup and delete.

5.2 Isolation expectation

Kind runs Kubernetes nodes as local Docker containers, but the scenarios deliberately request host access, privileged modes, host networking or elevated RBAC. The workstation and Docker daemon therefore remain part of the practical trust boundary.

6 Detection and event pipeline

DetectionMonitor runs concurrent Kubernetes watches for pods and ClusterRoleBindings. Matching resources become alert records, are retained in memory and are broadcast to connected clients. Five rule types are wired to the live watch handlers.

Rule	Severity	Detection condition
privileged-pod-creation	Critical	Container securityContext sets privileged true.
hostpath-mount	High	Pod declares a hostPath volume.
cluster-admin-binding	Critical	New binding references cluster-admin.
host-network	High	Pod requests hostNetwork.
host-pid	High	Pod requests hostPID.

6.1 Live events

WebSocketManager accepts clients at /ws and broadcasts JSON messages for attack starts, steps and completion; detection alerts; remediation tokens, commands and results; orchestrator state; and monitor lifecycle. Failed connections are removed during broadcast. The CLI uses a compatible local event sink to render the same sequence directly in the terminal.

6.2 State lifetime

Attack history, detection events and remediation sessions are held in process memory. The CLI separately writes engagement summaries to timestamped JSON files under results, which provide durable evidence after the process exits.

7 Claude remediation agent

For a selected incident, the agent builds context from attack metadata, affected infrastructure and up to five recent detection events. It streams Claude Sonnet 4 output, extracts completed thinking and command blocks, executes each discovered command and broadcasts the result before continuing. A summary block closes the session.

7.1 Command execution boundary

The executor tokenizes the model-provided text, accepts it when the first token is kubectl and runs it without a shell with a 30-second timeout. Output and return status are stored per remediation step. The implementation does not independently enforce the narrower verb and resource allowlist described in the prompt, so the configured cluster context and service account remain the effective authorization boundary.

7.2 Session record

Field	Meaning
session_id	Unique remediation identifier.
incident	Attack, severity, description, infrastructure and detection context.
steps	Analysis text, command, output, success state and timestamp.
status	Pending, running, completed or failed.
summary	Model-produced closeout when present.

8 CLI, API and security chat

8.1 CLI workflow

The primary interface offers ten single-scenario options, run-all, full engagement with remediation, result review and cluster status. It imports backend components in-process and formats analysis, commands, outputs, detections and summaries for the terminal. The --check flag performs a concise lab health check.

8.2 API groups

Group	Representative endpoints
Health and cluster	/api/health, /api/prerequisites, /api/cluster/info, create, setup and delete
Attacks	/api/attacks, run/{id}, run-all, active, history, result/{execution_id}, orchestrator
Detection	/api/detection/events, summary, start and stop
Remediation	/api/remediation/trigger, sessions, session detail and auto-trigger
Evidence	/api/results and /api/report
Model interface	/api/chat
Streaming	/ws WebSocket

8.3 Security chat

The chat handler assembles current attacks, alerts, cluster state, remediation records, MITRE data and affected infrastructure into model context. Claude returns a plain-text operational answer. This is an analytical interface to current in-memory state and does not itself execute Kubernetes commands.

9 Reporting and MITRE coverage

The ReportLab generator produces an assessment PDF containing engagement summary, attacks, severity, MITRE mappings, infrastructure details, detection alerts, remediation data and cluster context. The API returns the generated file from /api/report; the CLI result writer creates a complementary JSON record.

9.1 Coverage model

Attack metadata maps the ten modules to tactics including Privilege Escalation, Credential Access, Discovery, Collection and Impact, with related Defense Evasion, Lateral Movement and Execution techniques represented in the platform mapping. Coverage output is evidence that a scenario was exercised, not proof that every technique variant or detection source was validated.

9.2 Evidence interpretation

An attack result records start and end time, status, mapped technique, emitted events and infrastructure affected. A detection alert demonstrates a monitor rule match. A remediation step demonstrates that a kubectl command ran and records its return status; it does not by itself establish complete incident eradication.

10 Deployment and safe operation

10.1 Prerequisites

The supported workflow requires Python 3.11 or newer, Docker, Kind and kubectl. Backend dependencies include FastAPI, Uvicorn, websockets, the Kubernetes client, PyYAML, HTTPX, Anthropic and ReportLab. The React interface is installed separately from frontend/package.json.

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r backend/requirements.txt

# optional for remediation and chat
export ANTHROPIC_API_KEY=...

python3 cli.py
```

10.2 API and frontend

```
python3 backend/main.py

cd frontend
npm install
npm run dev
```

10.3 Operating controls

- Use a dedicated workstation or disposable environment and confirm kubectl points to the intended Kind context.
- Keep the Anthropic key out of source control and scope access to authorized operators.
- Delete the lab cluster after testing and review Docker resources that remain on the host.
- Treat live model output as untrusted input; review remediation authorization before enabling autonomous execution.

11 Implementation cross-reference

Source reference: [k8attack](#), revision [4585edce0737](#). The review covers source and configuration; it does not certify a live deployment or every runtime outcome.

This map ties the principal documented behaviors to their source modules.

Capability	Primary implementation
Attack base model and events	backend/attack_engine/attacks/base.py
Attack registry and execution	backend/attack_engine/engine.py; backend/attack_engine/orchestrator.py
Ten scenario implementations	backend/attack_engine/attacks/*.py
Kind cluster lifecycle and inventory	backend/cluster_manager/manager.py
Pod and RBAC detection watches	backend/detection/monitor.py
Claude remediation sessions	backend/remediation/agent.py
REST API and background workers	backend/main.py
WebSocket connections and broadcast	backend/ws_manager/handler.py
Security chat	backend/chat/handler.py
PDF assessment report	backend/report/generator.py
Terminal workflow and JSON results	cli.py
React monitoring interface	frontend/src/App.tsx; frontend/src/components/

12 Next steps

Priority	Improvement	Value
1	Enforce command policy in code	Validate kubectl verbs, resources, namespaces and flags before model-generated execution.
2	Add explicit operator approval	Place a review gate between generated remediation commands and cluster mutation.
3	Add integration tests	Verify each attack, alert rule, cleanup path and report against a disposable Kind cluster.
4	Persist service state	Retain attack, detection and remediation records across API restarts.
5	Tighten network boundaries	Bind development services deliberately and add authentication before multi-user exposure.